Seminar 6

The lecture gave you `map`, `filter`, `flatMap`, the folds and a few library combinators. Today we use them on problems the lecture did not show.

Rounds 1 and 2 drill the moves the homework needs. Rounds 3 and 4 are new: **folds that carry state** and **combinators that return `Option`**.
You will find both, under other names, in Kotlin, Python, JS and C++.

Round 1 — a list against its own tail

```
def adjacentPairs (xs : List α) : List (α × α) :=
  xs.zip xs.tail

def countRises (xs : List Int) : Nat :=
  (adjacentPairs xs).countP (fun (x, y) => x < y)
```

lining the list up with itself

```
xs       = [1, 3, 2, 5]
xs.tail  = [3, 2, 5]
zip      = [(1,3), (3,2), (2,5)]
rises    =   ✓      ·      ✓     -- 2
```

`zip` stops at the shorter list, so the last element pairs with nothing, and `[]` and `[x]` need no special case.

`runningMax`, ★ `localMaxima`

```
def runningMax : List Nat → List Nat
  | [] => []
  | x :: xs => xs.scanl max x

def localMaxima (xs : List Int) : List Int :=
  (List.zip xs (List.zip xs.tail xs.tail.tail)).filterMap
    fun (a, b, c) => if a < b ∧ c < b then some b else none
```

`scanl` keeps its starting value

```
[1, 4, 1, 5].scanl max 3
  = [3, 3, 4, 4, 5]   -- one longer than the input
```

Start from the head: the output is exactly as long as the input.

Round 2 — lists from ranges

```
def timesTable (n : Nat) : List (List Nat) :=
  (List.range n).map fun i =>
    (List.range n).map fun j => (i + 1) * (j + 1)

def divisors (n : Nat) : List Nat :=
  (List.range (n + 1)).filter fun d => 0 < d && n % d == 0

def perfectBelow (n : Nat) : List Nat :=
  (List.range n).filter fun m =>
    0 < m && (divisors m).sum == 2 * m
```

the shapes

```
-- map inside map   ↦ a table
-- filter on range  ↦ "all k < n with …"
-- divisors 6 = [1, 2, 3, 6], sum 12 = 2 * 6
```

`List.range n` starts at 0. Shift with `+ 1` or filter out 0. Do not hope that 0 behaves.

★ `twoSquares` — nested choices with `flatMap`

```
def twoSquares (n : Nat) : List (Nat × Nat) :=
  (List.range (n + 1)).flatMap fun a =>
    ((List.range (n + 1)).filter fun b =>
        1 ≤ a && a ≤ b && a * a + b * b == n)
      |>.map fun b => (a, b)
```

why `flatMap`, not `map`

```
-- each a gives a list of pairs
-- map     : List (List (Nat × Nat))
-- flatMap : List (Nat × Nat)
```

One `flatMap` per "for each", one `filter` per condition; output order follows the loops.

Round 3 (new) — a fold is a loop

```
def stats (xs : List Nat) : Nat × Nat × Nat :=
  xs.foldl (fun (count, total, top) x =>
    (count + 1, total + x, max top x)) (0, 0, 0)
```

the loop it replaces (Kotlin)

```
var count = 0; var total = 0; var top = 0
for (x in xs) {
  count += 1; total += x; top = maxOf(top, x)
}
// variables  ↦ one tuple: the state
// loop body  ↦ step : state → x → state
```

Several results from one pass: one tuple accumulator, not several folds.

`balanced` — a state that can break

```
def balanced (cs : List Char) : Bool :=
  let step : Option Nat → Char → Option Nat
    | some d, '(' => some (d + 1)
    | some (d + 1), ')' => some d
    | some 0, ')' => none
    | depth, _ => depth
  cs.foldl step (some 0) == some 0
```

`")("` step by step

```
some 0 —')'→ none —'('→ none
-- none never recovers: false
```

A bare depth counter accepts `")("`. The state needs an error value that never recovers.

`dedupAdjacent`, `balances` — what the state looks like

```
def dedupAdjacent [DecidableEq α] (xs : List α) : List α :=
  xs.foldr
    (fun x acc =>
      match acc with
      | y :: _ => if x = y then acc else x :: acc
      | [] => [x])
    []

def balances (start : Int) (moves : List Int) : List Int :=
  moves.scanl (· + ·) start
```

`foldr` sees the processed rest

```
1 :: [1, 2]   -- 1 = 1, keep acc
2 :: [3]      -- new run, cons
```

`foldr` lets the step `match` on what is already built. `scanl` is `foldl` that keeps every state it passed through.

`foldl_add_start` — the start must vary

```
theorem foldl_add_start (a : Nat) (xs : List Nat) :
    xs.foldl (· + ·) a = a + xs.sum := by
  induction xs generalizing a with
  | nil => simp
  | cons x xs ih => simp [ih]; omega
```

the `cons` goal

```
ih : ∀ (a : ℕ), List.foldl (fun x1 x2 => x1 + x2) a xs
                    = a + xs.sum
⊢ List.foldl (fun x1 x2 => x1 + x2) a (x :: xs)
    = a + (x :: xs).sum
```

A `foldl` step changes the start to `a + x`, so the IH must hold for every start.

Round 4 (new) — search, don't loop

Lean	returns	Kotlin	Python
`xs.find? p`	first match, or `none`	`firstOrNull(p)`	`next((x for x in xs if p(x)), None)`
`xs.findIdx? p`	its position	`indexOfFirst(p)` (−1)	`next((i for i, x in enumerate(xs) if p(x)), None)`
`xs.filterMap f`	the `some` results	`mapNotNull(f)`	`[y for x in xs if (y := f(x)) is not None]`
`xs.lookup k`	value of the first `(k, v)`	`firstOrNull { it.first == k }?.second`	`next((v for k2, v in xs if k2 == k), None)`
`xs.takeWhile p`, `dropWhile p`	prefix / the rest	same names	`itertools.takewhile`, `dropwhile`
`xs.span p`	both at once	—	—
`xs.partition p`	(pass, fail)	`partition(p)`	two comprehensions
`xs.splitBy r`	runs of related neighbours	—	`itertools.groupby`
`xs.head?`, `xs.max?`	`none` on `[]`	`firstOrNull()`, `maxOrNull()`	`max(xs, default=None)`
`o.getD d`	unwrap with a default	`?: d`	`o if o is not None else d`

Instead of −1 or an exception, the type says "might be missing", and the caller has to handle that case.

`digitsIn`, `callerName`

```
def digitsIn (cs : List Char) : List Nat :=
  cs.filterMap fun c =>
    if c.isDigit then some (c.toNat - '0'.toNat) else none

def callerName (book : List (String × String))
    (number : String) : String :=
  (book.lookup number).getD "unknown"
```

one pass where you would write two

```
-- filter isDigit, then map value
--   = filterMap (one function decides both)
-- lookup gives the FIRST entry: "Ada", not "Grace"
```

`filterMap` fuses "keep it?" and "turn it into what?" into one function, which suits a parse that can fail per element.

`splitPassFail`, `firstOverdraft?`

```
def splitPassFail (threshold : Nat)
    (results : List (String × Nat)) : List String × List String :=
  let (pass, fail) := results.partition fun r => threshold ≤ r.2
  (pass.map (·.1), fail.map (·.1))

def firstOverdraft? (start : Int) (moves : List Int) : Option Nat :=
  (balances start moves).findIdx? (· < 0)
```

reusing round 3

```
balances 10 [-3, 5, -20, 30, -40]
  = [10, 7, 12, -8, 22, -18]
--                ↑ index 3
```

Build the history with `scanl`, then search it with `findIdx?`.

`find?_eq_head?_filter`, ★ `takeWhile_append_dropWhile`

```
theorem find?_eq_head?_filter (p : α → Bool) (xs : List α) :
    xs.find? p = (xs.filter p).head? := by
  induction xs with
  | nil => rfl
  | cons x xs ih =>
      cases h : p x <;> simp only [List.find?, List.filter, h, ih, List.head?_cons]

theorem takeWhile_append_dropWhile (p : α → Bool) (xs : List α) :
    takeWhile p xs ++ dropWhile p xs = xs := by
  induction xs with
  | nil => rfl
  | cons x xs ih => cases h : p x <;> simp [takeWhile, dropWhile, h, ih]
```

Every definition here branches on `p x`, so the proof branches on it too: `cases h : p x` keeps the equation `h` that lets `simp` pick the right branch.

The same vocabulary elsewhere

Lean	Kotlin	Python	JS	C++
`map f`	`map`	`map`, comprehension	`map`	`std::views::transform`
`filter p`	`filter`	`filter`, comprehension	`filter`	`std::views::filter`
`flatMap f`	`flatMap`	nested comprehension	`flatMap`	`views::transform | views::join`
`foldl f z`	`fold(z, f)`	`functools.reduce(f, xs, z)`	`reduce(f, z)`	`std::accumulate`
`foldr f z`	`foldRight(z, f)`	—	`reduceRight`	`std::accumulate` on reversed range
`scanl f z`	`runningFold(z, f)`	`itertools.accumulate`	—	`std::inclusive_scan`
`zip`, `zipWith`	`zip`, `zip { }`	`zip`	—	`std::views::zip` (C++23)
`find?`, `filterMap`	`firstOrNull`, `mapNotNull`	`next(…, None)`, comprehension	`find` (`undefined`)	`std::ranges::find_if`
`takeWhile`, `partition`	same names	`itertools.takewhile`, —	—	`views::take_while`, `std::partition_copy`